

University of Science and Technology of Hanoi

CyberSec Internship – Weekly Reporting

## Week 1 Report

### Wazuh Architecture, Log Collection, Data Flow, and Rule-Based Detection

Expanded revision covering all-in-one, single-node, and multi-node deployment models; cross-platform log sources; cloud integrations; and the current internship custom ruleset

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <b>Student</b>         | Bui Tat Dat                                      |
| <b>Branch</b>          | bui-tat-dat                                      |
| <b>Report folder</b>   | week1_31-03-2026                                 |
| <b>Issue topic</b>     | Research Wazuh architecture & detection workflow |
| <b>Submission date</b> | 01-04-2026                                       |

## Contents

|          |                                                                                      |           |
|----------|--------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Contribution Scope</b>                                                            | <b>2</b>  |
| <b>2</b> | <b>Objective of This Week</b>                                                        | <b>2</b>  |
| <b>3</b> | <b>Detailed Work</b>                                                                 | <b>2</b>  |
| 3.1      | Description . . . . .                                                                | 2         |
| 3.2      | Design / Method . . . . .                                                            | 3         |
| 3.3      | Implementation . . . . .                                                             | 3         |
| <b>4</b> | <b>Results / Output</b>                                                              | <b>3</b>  |
| 4.1      | Architecture overview and deployment models . . . . .                                | 3         |
| 4.1.1    | Endpoint and Wazuh agent . . . . .                                                   | 5         |
| 4.1.2    | Wazuh server . . . . .                                                               | 5         |
| 4.1.3    | Wazuh indexer . . . . .                                                              | 5         |
| 4.1.4    | Wazuh dashboard . . . . .                                                            | 6         |
| 4.2      | Log collection across endpoints, applications, devices, and cloud services . . . . . | 6         |
| 4.2.1    | Cross-platform endpoint collection . . . . .                                         | 6         |
| 4.2.2    | Applications, web services, WAFs, and network devices . . . . .                      | 7         |
| 4.2.3    | Cloud and SaaS logs . . . . .                                                        | 8         |
| 4.2.4    | Current internship data sources . . . . .                                            | 9         |
| 4.3      | Data flow from collection to analysis and alerting . . . . .                         | 10        |
| <b>5</b> | <b>Analysis</b>                                                                      | <b>11</b> |
| 5.1      | How Wazuh applies decoders and rules . . . . .                                       | 11        |
| 5.2      | Applied case study: the current custom rules . . . . .                               | 12        |
| 5.2.1    | Example 1: application brute-force correlation . . . . .                             | 13        |
| 5.2.2    | Example 2: SQL injection and command-injection detection . . . . .                   | 14        |
| 5.3      | How analysis becomes alerts . . . . .                                                | 14        |
| <b>6</b> | <b>Reusable Content</b>                                                              | <b>14</b> |
| 6.1      | Reusable Content Part I: Wazuh architecture and deployment models . . . . .          | 15        |
| 6.2      | Reusable Content Part II: Cross-platform and multi-source log collection . . . . .   | 15        |
| 6.3      | Reusable Content Part III: Rule-based detection workflow . . . . .                   | 15        |
| <b>7</b> | <b>Next Step</b>                                                                     | <b>15</b> |

## 1. Contribution Scope

This week contributes the architectural foundation required before deeper implementation, tuning, and validation. The scope includes four connected layers. The first layer is architecture: understanding the Wazuh agent, Wazuh server, Wazuh indexer, Wazuh dashboard, and the official deployment models. The second layer is collection: understanding how Wazuh receives security telemetry from Linux, Windows, macOS, applications, network devices, and cloud services. The third layer is data flow: understanding the sequence from raw log generation to decoding, rule evaluation, alert generation, indexing, and dashboard visualization. The fourth layer is detection logic: understanding how Wazuh decoders and XML rules operate, then validating that logic against the current internship `local_rules.xml`.

## 2. Objective of This Week

- Understand the official Wazuh architecture composed of the Wazuh agent and the three central components: Wazuh server, Wazuh indexer, and Wazuh dashboard [1, 5].
- Distinguish the official all-in-one, single-node, and multi-node deployment models and relate them to different scale and availability requirements [1, 3, 4].
- Explain how Wazuh collects logs from Linux, Windows, macOS, network devices, applications, and cloud services such as AWS, Azure, Google Cloud, Office 365, and GitHub [6, 8, 17, 18, 19, 20, 21, 22].
- Describe the event path *log -> decode -> rule match -> alert -> index -> dashboard* and identify which component performs each function [1, 10, 26].
- Analyze how Wazuh rule-based detection works using both official rule syntax and the current internship custom rules [23, 24, 25].

## 3. Detailed Work

### 3.1 Description

Wazuh is structured around one distributed sensing layer and three central processing layers. The sensing layer is the Wazuh agent, which runs on monitored endpoints and forwards telemetry to the Wazuh server. The central components are the Wazuh server, the Wazuh indexer, and the Wazuh dashboard. The Wazuh server receives events, decodes them, evaluates rules, and generates alerts. The Wazuh indexer stores alert and event data for search and analytics. The Wazuh dashboard provides visualization, filtering, reporting, and management capabilities [1, 5].

Beyond static architecture, the platform must also be understood from the viewpoint of source coverage. Wazuh is not limited to one operating system or one type of log. The official documentation states that Wazuh collects, analyzes, and stores logs from endpoints, network devices, and applications, and can also receive events by syslog or API-based integrations [6, 7]. This makes the architecture meaningful only when it is connected to the actual intake paths of Linux, Windows, macOS, network-device, application, and cloud activity.

### 3.2 Design / Method

The report follows a documentation-to-environment method. First, official Wazuh documentation was reviewed to extract the standard roles of the agent, server, indexer, and dashboard, together with deployment options and the standard log-analysis workflow. Second, the current internship environment was examined through the existing `ossec.conf` and the current `local_rules.xml`. Third, the architecture discussion was split into two complementary views: a deployment-topology view and a component-level logical view. Fourth, the log-collection section was expanded to classify sources by operating system, application or network role, and cloud integration. Fifth, the rule-analysis section was written to connect official decoder and rule syntax with the current custom rules used in the lab.

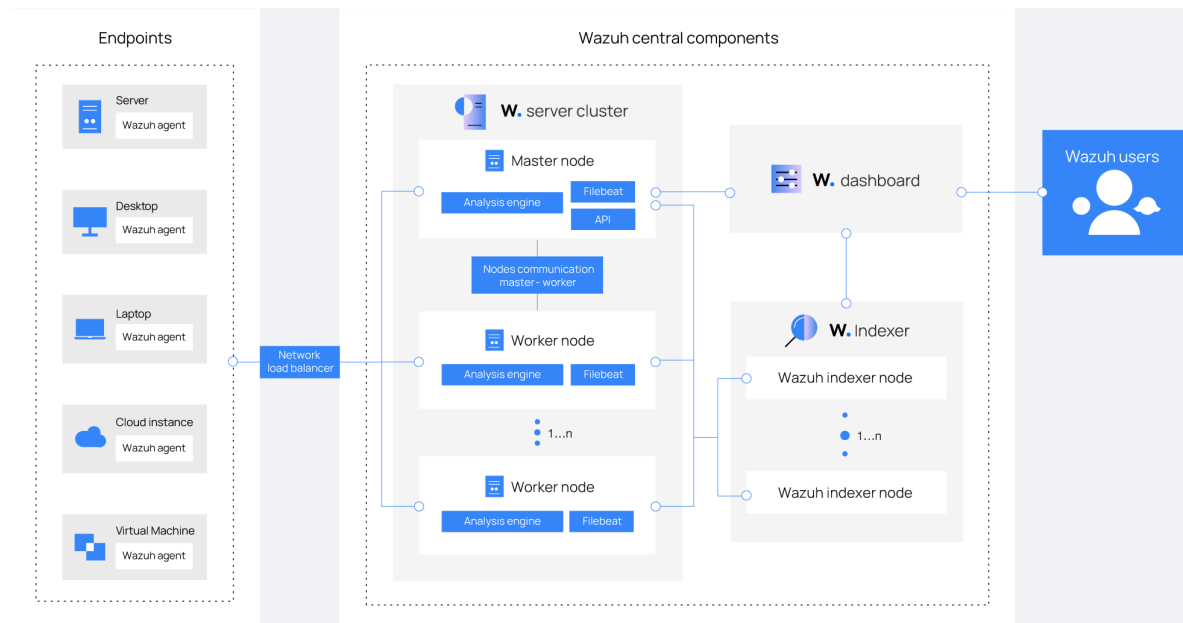
### 3.3 Implementation

Week 1 was assigned as a research task rather than an installation task, but the report still delivers implementation-oriented outputs. These outputs are: (i) an official Wazuh deployment architecture figure, (ii) a component-level architecture figure attributed to the external source provided by the user, (iii) a comparison table of all-in-one, single-node, and multi-node deployments, (iv) a cross-platform log source matrix, (v) a single-node event-processing figure, (vi) a table of the current `ossec.conf` sources, and (vii) a technical analysis of the current local rules.

## 4. Results / Output

### 4.1 Architecture overview and deployment models

The official Wazuh architecture documentation defines three central components — server, indexer, and dashboard — plus the Wazuh agent on endpoints. It also distinguishes three deployment models. In an **all-in-one deployment**, all central components are installed on the same host, which is suitable for labs and small environments. In a **single-node deployment**, the server, indexer, and dashboard are each placed on separate hosts, which improves performance for medium environments. In a **multi-node deployment**, one dashboard is typically combined with multiple server nodes and multiple indexer nodes to provide higher throughput, scalability, and high availability for larger environments [1, 3, 4].

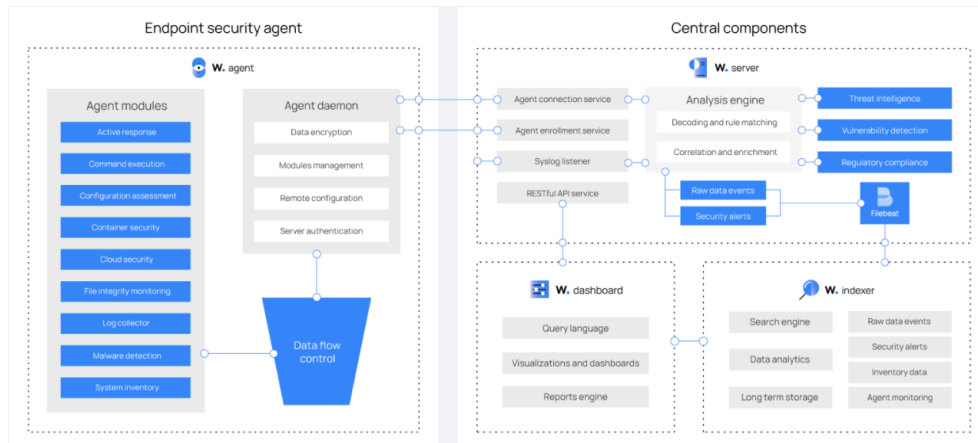


**Figure 1:** Official Wazuh deployment architecture used as the topology baseline.

Source: Wazuh Documentation, *Architecture*, current documentation page. The image emphasizes clustered Wazuh server and Wazuh indexer roles and therefore represents the scalable reference topology rather than one fixed lab layout [1].

**Table 1:** Comparison of the official Wazuh deployment models

| Model       | Physical layout                                                           | When it fits best                                                                                  | Key architectural note                                                                                  |
|-------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| All-in-one  | Server, indexer, and dashboard on one host                                | Labs, proofs of concept, and small environments with limited monitored endpoints                   | Simplest to deploy and operate, but all central workloads share one machine [1, 2].                     |
| Single-node | One server node, one indexer node, one dashboard node on separate hosts   | Medium environments that need more separation and performance than all-in-one                      | Logical roles are the same, but compute and storage are separated across three hosts [1].               |
| Multi-node  | One dashboard plus multiple Wazuh server nodes and multiple indexer nodes | Large environments with high event throughput, redundancy needs, or high availability requirements | Uses clustering and often load balancing so agent traffic and processing can scale horizontally [1, 4]. |



**Figure 2:** Component-level Wazuh architecture used to explain functional roles.

*Source:* Screenshot provided by the user and attributed to inLab FIB, *Wazuh – An Open Source platform unifying SIEM and XDR*, published 27 May 2024. This image is used here as a logical architecture view rather than as an exact node-by-node deployment map [28].

Figure 2 is useful because it explains *what each component does*, whereas Figure 1 explains *how those components can be deployed at scale*. The component-level view is therefore the best figure for describing endpoint behavior, server-side analysis, indexing, and dashboard interaction.

#### 4.1.1 Endpoint and Wazuh agent

The endpoint side of Figure 2 separates the Wazuh agent into agent modules and the agent daemon. This is a useful conceptual distinction. The agent modules correspond to sensing and security capabilities such as log collection, file integrity monitoring, system inventory, configuration assessment, malware-related monitoring, command execution, and active response. The official Wazuh agent documentation confirms that the agent ships collected data and security-related events to the server and also reports operational information such as configuration and status [5]. In practical terms, the endpoint is not a passive forwarder; it is an active sensor that can collect logs, state, and security context from the local host.

#### 4.1.2 Wazuh server

The Wazuh server is the analytical core of the platform. The official architecture documentation states that agents send events to the server on TCP port 1514 by default, and that the server decodes events and matches rules against them using the Wazuh analysis engine [1]. The server installation guide further states that the Wazuh server includes the Wazuh manager and Filebeat [3]. In functional terms, the server receives data from agents, syslog-enabled devices, and API-based integrations; normalizes events through decoders; applies rule-based detection; correlates repeated behavior; generates alerts; and forwards output to the indexer.

#### 4.1.3 Wazuh indexer

The Wazuh indexer is the storage, search, and analytics layer. The dashboard documentation describes the indexer as a real-time, full-text search and analytics engine for security data, and

states that log data ingested into the Wazuh server is analyzed and then forwarded to the indexer for indexing and storage [27]. Architecturally, the indexer is what turns server output into searchable historical security records. It stores security alerts, raw or archived event data, inventory-related data, and agent monitoring information, enabling both short-term triage and longer-term retrospective analysis.

#### 4.1.4 Wazuh dashboard

The Wazuh dashboard is the analyst-facing layer. The official architecture page states that, once data has been indexed, the dashboard is used to query and visualize security information; the dashboard also interacts with the Wazuh server API for status and configuration-related views [1]. This means the dashboard has a dual role. First, it provides dashboards, searches, and reports over indexed security data. Second, it acts as an administrative interface for agents, ruleset testing, and platform status inspection.

### 4.2 Log collection across endpoints, applications, devices, and cloud services

Wazuh documentation states that the platform collects, analyzes, and stores logs from endpoints, network devices, and applications, and that log messages can be sent to the Wazuh server by agents, syslog, or third-party API integrations [6, 7]. This makes log collection a core architectural topic rather than a secondary configuration detail.

#### 4.2.1 Cross-platform endpoint collection

The official log-analysis use case documentation gives clear examples of operating-system coverage. Wazuh can collect syslog, auditd, application logs, and other logs from Linux endpoints. On Windows, the Wazuh agent collects Windows events through the Windows event channel and Windows event log format, monitoring the System, Application, and Security channels by default, while allowing additional channels to be configured. On macOS, Wazuh can collect logs through the macOS unified logging system [6, 8, 12]. This means endpoint coverage is fundamentally multi-platform.

**Table 2:** Typical endpoint log sources by operating system

| Platform | Typical sources                                                                                                    | How Wazuh collects them                                                                                                                                                                                             |
|----------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linux    | syslog, audit.log / auditd, application logs, web logs, package logs, command outputs, journals                    | Mostly through the agent <localfile> mechanism, Logcollector, command monitoring, or syslog forwarding; auditd can be mapped into security-relevant fields and rules [6, 9, 13].                                    |
| Windows  | Event Log channels such as System, Application, Security, plus additional channels like Sysmon or Windows Defender | Through the Windows event channel / event log format. By default the agent monitors System, Application, and Security, and can be extended to monitor other channels such as Sysmon-based telemetry [6, 8, 14, 15]. |
| macOS    | Unified log records, host telemetry, application events                                                            | Through the Wazuh agent using the macOS unified logging system, allowing host activity on macOS endpoints to be brought into the same analysis engine [12].                                                         |

#### 4.2.2 Applications, web services, WAFs, and network devices

At application and network level, Wazuh can monitor file-based logs, JSON logs, web-server logs, WAF events, and syslog traffic. The official `localfile` reference states that this configuration block is used to collect data from files, Windows events, and command outputs [9]. The JSON decoder documentation explains that Wazuh has a default decoder for JSON logs and can extract JSON data types without requiring a custom decoder for every JSON source [16]. This is highly relevant for application logs and WAF logs, because modern web-security components often emit JSON.



**Table 3:** Common non-endpoint log families relevant to the internship use case

| Log family                      | Examples                                                                                           | Architectural intake path                                                                                                                                     |
|---------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Web access and error logs       | <code>access.log</code> , <code>error.log</code> , reverse-proxy logs, API gateway logs            | Collected by the Wazuh agent from files on Linux or by centralized log shipping from the monitored service host [7, 9].                                       |
| WAF / security-device logs      | ModSecurity audit logs, WAF JSON events, IDS or IPS alerts                                         | Collected as local files if hosted on the monitored server, or forwarded via syslog or API integrations when generated by external security devices [16, 11]. |
| Firewall / router / switch logs | Firewall denies, VPN events, router configuration events, switch syslog output                     | Collected through remote syslog on the Wazuh server from devices that do not support agent installation [11].                                                 |
| Application logs                | Custom <code>app.log</code> , microservice JSON logs, authentication events, business-logic events | Collected through <code>localfile</code> and decoded by built-in or custom decoders, then matched by built-in or custom rules [9, 16].                        |

#### 4.2.3 Cloud and SaaS logs

Wazuh's cloud-security documentation states that the platform collects and analyzes log data from AWS, Azure, Google Cloud, Office 365, and GitHub [17]. The cloud-specific documentation explains this in more detail: Wazuh can monitor AWS through agents on EC2 instances or supported AWS-service integrations; it provides solutions for collecting and analyzing security and runtime events from Microsoft Azure; it can monitor Google Cloud through agents on instances or supported cloud-service modules; it supports Office 365 audit-log monitoring for user and administrative activity; and it can monitor GitHub audit logs for organization, permission, membership, and repository events [18, 19, 20, 21, 22].

**Table 4:** Cloud and SaaS sources mentioned in the official Wazuh documentation

| Source       | Typical activity or log focus                                                                     | How it fits the Wazuh architecture                                                                                         |
|--------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| AWS          | EC2 activity, AWS service logs, configuration anomalies, unauthorized behavior, S3-related events | Collected through agents on cloud instances and AWS integrations that feed event data into the Wazuh analysis engine [18]. |
| Azure        | Security and runtime events from Azure infrastructure and supported services                      | Collected through Azure-oriented integrations and analyzed in the same server-side decoder and rule pipeline [19].         |
| Google Cloud | Instance telemetry plus supported Google Cloud service events                                     | Collected through agents or cloud modules so that anomalies can trigger near real-time alerts [20].                        |
| Office 365   | Audit activity for Exchange, SharePoint, OneDrive, Teams, and related administrative actions      | Collected through Office 365 audit-log integration for user and admin visibility [21].                                     |
| GitHub       | Organization audit logs, permission changes, member changes, Git-related activity                 | Collected through GitHub audit-log monitoring to detect suspicious administrative or repository activity [22].             |

#### 4.2.4 Current internship data sources

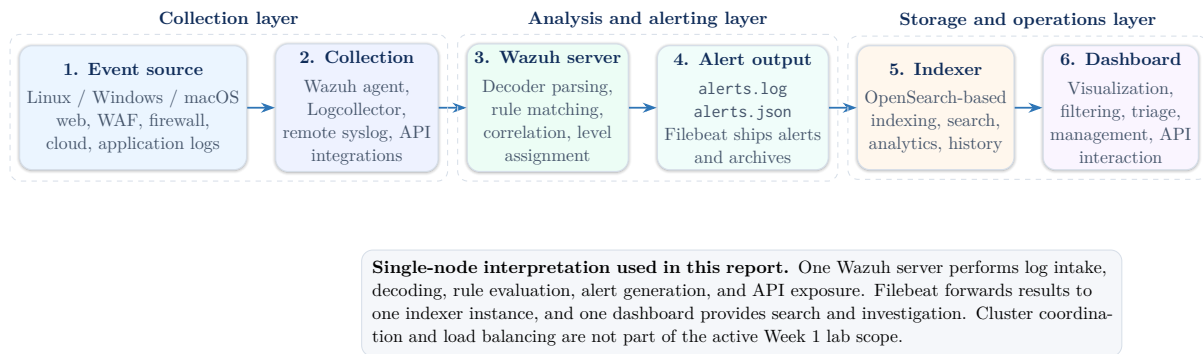
The current internship `ossec.conf` confirms that the present environment already follows this multi-source design on a smaller scale. The current agent `weburban` connects to the Wazuh server at `192.168.1.13:1514/TCP`. It collects `Nginx access.log` and `error.log`, a JSON application log, a JSON ModSecurity audit log, `/var/log/dpkg.log`, the active-responses log, three command outputs, and several host-security modules such as Rootcheck, Syscheck, Syscollector, and SCA. This means the lab already combines host telemetry, web logs, application events, and WAF events in one collection pipeline.

**Table 5:** Main data sources in the current internship `ossec.conf`

| Source type               | Format / interval                    | Purpose in the detection workflow                                                                       |
|---------------------------|--------------------------------------|---------------------------------------------------------------------------------------------------------|
| Agent connection          | TCP 1514                             | Ships endpoint data from the agent to the Wazuh server.                                                 |
| Rootcheck                 | Every 12h                            | Looks for rootkits, suspicious files, devices, ports, interfaces, and related host indicators.          |
| Syscollector              | Every 1h                             | Collects hardware, OS, network, package, process, port, service, user, and browser-extension inventory. |
| SCA                       | Every 12h                            | Runs security configuration assessment policies.                                                        |
| Syscheck                  | Every 12h + synchronization every 5m | Performs file integrity monitoring on critical directories.                                             |
| Command outputs           | Every 360s                           | Collects filesystem usage, listening-port information, and recent login records.                        |
| Nginx access / error logs | apache parser                        | Provides HTTP request and server-error telemetry.                                                       |
| Active responses log      | syslog                               | Records automated response actions taken by Wazuh.                                                      |
| Package-management log    | syslog                               | Tracks package installation or update activity via <code>/var/log/dpkg.log</code> .                     |
| Application log           | JSON                                 | Provides custom application events from <code>/home/datbui/grp/web/app.log</code> .                     |
| ModSecurity audit log     | JSON                                 | Provides web application firewall telemetry from <code>/var/log/modsec_audit.log</code> .               |

### 4.3 Data flow from collection to analysis and alerting

The official Wazuh log-collection workflow states that Logcollector gathers logs from endpoints, applications, and network devices, and that the Wazuh server analyzes collected logs in real time using decoders and rules. It further states that the Analysisd module evaluates decoded logs against rules and records alerts in `/var/ossec/logs/alerts/alerts.log` and `/var/ossec/logs/alerts/alerts.json` [10, 26]. The architecture documentation adds that Filebeat forwards alert and event data to the Wazuh indexer, after which the dashboard is used to query and visualize security information [1, 3].



**Figure 3:** Single-node event processing and alert pipeline drawn directly in LaTeX/TikZ.

*Source:* Author synthesis based on Wazuh architecture, log-data collection, and server-installation documentation [1, 10, 3].

The processing sequence in Figure 3 can be described in seven stages:

1. A source generates a security-relevant event. This source may be a Linux, Windows, or macOS endpoint, a web server, a WAF, a firewall, or a cloud integration.
2. The Wazuh agent, remote syslog listener, or API integration collects or forwards the event to the Wazuh server [6, 11, 17].
3. The Wazuh server applies decoders to transform the raw event into structured fields [23].
4. The Wazuh ruleset evaluates the decoded fields. If the rule conditions are met, the server assigns an alert level and writes alert output [24, 26].
5. Alerts and archived event data are forwarded by Filebeat to the Wazuh indexer using the server-to-indexer path documented in the official architecture guide [1, 3].
6. The Wazuh indexer stores the data for search, analytics, and historical retrieval [27].
7. The Wazuh dashboard queries indexed data and the server API so analysts can review, filter, correlate, and manage the resulting security information [1].

**Where alerts appear.** According to the official alert-management documentation, alerts are generated by the Wazuh manager after event processing and are stored by default in `/var/ossec/logs/alerts/alerts.log` and `/var/ossec/logs/alerts/alerts.json`. The minimum level for storing alerts is controlled by `<log_alert_level>`, whose default value is 3 [26]. This clarifies that the analysis decision happens on the server before indexed visualization happens on the dashboard.

## 5. Analysis

### 5.1 How Wazuh applies decoders and rules

The decoder documentation states that decoders extract information from received events and separate that information into blocks to prepare the event for subsequent analysis [23]. The

rules syntax documentation states that the Wazuh ruleset, combined with custom rules, analyzes incoming events and generates alerts when all specified conditions in a rule are met [24]. In other words, Wazuh detection is not a black box. It is a structured pipeline:

$$\text{raw event} \xrightarrow{\text{decoder}} \text{fields} \xrightarrow{\text{ruleevaluation}} \text{alert level} + \text{description}$$

A rule can contain conditions on fields, URLs, programs, regex patterns, or previous rule matches. The official rules syntax guide states that each rule has an `id` and a `level`, and that the level is used by alerts and responses. The same guide also documents `frequency`, `timeframe`, `if_sid`, and `if_matched_sid`, which are essential for expressing correlation logic across repeated events [24]. For JSON sources, Wazuh provides a built-in JSON decoder that can extract strings, numbers, booleans, null values, arrays, and objects from JSON logs [16]. This is particularly important for application logs and WAF telemetry.

**Table 6:** Key Wazuh rule elements used in this research report

| Element                                                       | Meaning                                             | Why it matters in practice                                                                   |
|---------------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>id</code>                                               | Unique rule identifier                              | Lets one rule refer to another and helps analysts identify which logic fired [24].           |
| <code>level</code>                                            | Severity assigned to a match                        | Controls alert seriousness and interacts with alert storage and response logic [24, 26].     |
| <code>decoded_as</code>                                       | Requires a specific decoder format, such as JSON    | Useful when rules should only run after a particular parsing stage [16].                     |
| <code>field, url, match, regex</code>                         | Conditions on extracted or raw values               | Enables content-based detection of SQLi, command execution, or suspicious web patterns [24]. |
| <code>if_sid</code>                                           | Requires a previous rule match                      | Allows rule chaining and refinement of earlier parsing or matching stages [24].              |
| <code>if_matched_sid, frequency, timeframe, same_field</code> | Expresses temporal correlation over repeated events | Useful for brute-force, scanning, or repeated-failure patterns [24].                         |

## 5.2 Applied case study: the current custom rules

The current `local_rules.xml` demonstrates how the official rule syntax is adapted to the internship environment. The present ruleset contains custom detections for suspicious POST body content, SQL injection indicators in URLs and JSON fields, command-injection indicators in JSON application events, and application brute-force behavior. This is a strong fit for the current source coverage because the agent already collects Nginx logs, JSON application logs, and JSON ModSecurity audit logs.

**Table 7:** Summary of the current custom rules used in the internship environment

| Rule ID | Event family      | Level | Technical meaning                                                                                                                                                                            |
|---------|-------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 100005  | Nginx POST body   | 12    | Detects command-execution keywords such as <code>base64</code> , <code>bash</code> , <code>python</code> , <code>php</code> , <code>nc</code> , and <code>socat</code> in POST body content. |
| 110001  | Web URL SQLi      | 13    | Detects SQLMap-style blind SQL injection indicators in the decoded URL or query string.                                                                                                      |
| 110002  | JSON app SQLi     | 13    | Detects SQL injection indicators inside the JSON field <code>email</code> .                                                                                                                  |
| 110003  | JSON app RCE      | 13    | Detects suspicious command tokens in the JSON field <code>targetIp</code> for event <code>admin_ping</code> .                                                                                |
| 120001  | App login failure | 5     | Flags a single application login failure event.                                                                                                                                              |
| 120010  | App brute force   | 10    | Correlates five <code>login_failed</code> events from the same IP in 60 seconds.                                                                                                             |

These rules illustrate four design ideas.

- **Field-based matching.** SQL injection and RCE rules target decoded values such as `url`, `email`, and `targetIp`.
- **JSON-aware detection.** The `decoded_as=json` requirement ensures that rules are applied only after JSON parsing has succeeded.
- **Rule chaining.** The Nginx rule uses `if_sid` to depend on upstream web-log decoding logic.
- **Temporal correlation.** The application brute-force rule uses `if_matched_sid`, `frequency`, `timeframe`, and `same_field` to detect repeated failures from the same source.

### 5.2.1 Example 1: application brute-force correlation

The pair 120001 and 120010 is a direct example of Wazuh correlation logic. The first rule records a single failed login event. The second rule waits for repeated matches of the first rule and escalates the situation when five failures occur in sixty seconds from the same IP. Architecturally, this demonstrates that Wazuh detection is stateful enough to express short-window attack patterns, not just one-off signatures.

**Listing 1:** Custom application brute-force logic from the current environment

```
<rule id="120001" level="5">
  <field name="event">login_failed</field>
  <description>App login failed from IP %ip</description>
  <group>app,authentication</group>
</rule>

<rule id="120010" level="10" frequency="5" timeframe="60">
  <if_matched_sid>120001</if_matched_sid>
  <same_field>ip</same_field>
```

```
<description>App brute force detected from IP %ip</description>
<group>app,bruteforce</group>
</rule>
```

### 5.2.2 Example 2: SQL injection and command-injection detection

The custom rules also show how Wazuh is adapted to application-layer detection. Rule 110001 targets URL-based SQL injection patterns. Rule 110002 checks the JSON field `email` for SQLi indicators, which is useful when malicious payloads are hidden inside application parameters rather than raw URLs. Rule 110003 checks whether the JSON field `targetIp` contains command-execution tokens rather than a valid IP-like value, which is a practical way to detect command-injection attempts in application logic.

**Listing 2:** Two custom application-layer rules used in the current environment

```
<rule id="110002" level="13">
  <decoded_as>json</decoded_as>
  <field name="email" type="pcre2">(?(i)(information_schema|ord\s*\(|mid\s*\(|ifnull|
    select\s+case|union\s+select)</field>
  <description>Application: SQL Injection detected in Email field</description>
  <mitre><id>T1190</id></mitre>
</rule>

<rule id="110003" level="13">
  <decoded_as>json</decoded_as>
  <field name="event">admin_ping</field>
  <field name="targetIp" type="pcre2">(?(i)(base64|bash|sh|ping|whoami|cat|nc|socat)</
    field>
  <description>Application: RCE Attempt Detected</description>
  <mitre><id>T1059</id></mitre>
</rule>
```

## 5.3 How analysis becomes alerts

Wazuh analysis becomes an operational alert only after rule conditions are met on the server. This matters because collection alone is not enough. A Linux `audit.log` entry, a Windows Event Log record, a macOS unified log entry, a firewall syslog message, an Office 365 audit event, or a GitHub audit event all become meaningful security output only after the server has decoded the record and matched it to a rule or rule chain. The official log-analysis documentation states that Wazuh collects and analyzes logs from endpoints, network devices, and applications, while the cloud-security documentation states that Wazuh also collects and analyzes logs from AWS, Azure, Google Cloud, Office 365, and GitHub [6, 17]. Therefore, a unified alerting pipeline is one of Wazuh's most important architectural advantages: heterogeneous input sources are normalized into one server-side detection model.

## 6. Reusable Content

## 6.1 Reusable Content Part I: Wazuh architecture and deployment models

Wazuh should be documented as a four-part architecture consisting of the endpoint agent, Wazuh server, Wazuh indexer, and Wazuh dashboard. The agent is the distributed sensing component deployed on monitored systems. The server is the analytical component that performs event intake, decoding, rule matching, correlation, alert generation, and API exposure. The indexer is the storage and search component that retains alert and event data as searchable records. The dashboard is the visualization and management component used by analysts and administrators [1, 5, 27].

The same logical architecture can be packaged into different deployment models depending on scale. In all-in-one mode, all central components are installed on one host, which is convenient for labs and small environments. In single-node mode, the three central components are separated onto different hosts, improving performance isolation. In multi-node mode, the server and indexer are clustered to provide scalability and availability. A good final report should therefore distinguish between *logical architecture* and *deployment topology*, because the component roles remain stable even when the physical packaging changes [1, 4].

## 6.2 Reusable Content Part II: Cross-platform and multi-source log collection

Wazuh log collection is broad enough to unify endpoint, application, network, and cloud telemetry in one platform. On Linux, it can collect syslog, auditd data, application logs, package logs, and other file-based sources. On Windows, it collects the System, Application, and Security channels by default and can be extended to channels such as Sysmon or Windows Defender. On macOS, it can use the unified logging system. Beyond endpoints, Wazuh can ingest firewall and router syslog, WAF logs, custom application logs, and cloud-service activity from AWS, Azure, Google Cloud, Office 365, and GitHub [6, 8, 11, 17, 18, 19, 20, 21, 22]. This breadth of supported sources is one of the main reasons Wazuh is useful as a SIEM-oriented monitoring platform.

## 6.3 Reusable Content Part III: Rule-based detection workflow

Wazuh rule-based detection begins after collection. Decoders first parse raw logs and extract fields such as URLs, source IP addresses, usernames, JSON keys, or Windows event values. Rules then evaluate those extracted values. A simple rule can detect a suspicious pattern in one event, while more advanced rules can depend on previous rule matches or correlate repeated behavior over time using `if_sid`, `if_matched_sid`, `frequency`, `timeframe`, and `same_field` [23, 24]. The internship ruleset demonstrates this well through URL-based SQL injection detection, JSON-field SQL injection detection, JSON-based command-injection detection, and correlated application brute-force detection.

## 7. Next Step

The next step should move from architecture research to controlled validation. A strong Week 2 continuation would include testing the current custom rules against representative logs by using `/var/ossec/bin/wazuh-logtest` or the dashboard ruleset test, validating that the expected decoder fields are extracted, and checking how alert levels and rule IDs appear in the dashboard. After that, the internship work can expand in two directions: first, tune the current web and



application detections to reduce false positives; second, broaden the collection scope by adding new sources such as Windows event logs or Sysmon telemetry, firewall syslog, or cloud-audit sources.

## References

- [1] Wazuh Documentation. *Architecture*. Current documentation. Available at: <https://documentation.wazuh.com/current/getting-started/architecture.html>. Accessed 1 April 2026.
- [2] Wazuh Documentation. *Quickstart*. Current documentation. Available at: <https://documentation.wazuh.com/current/quickstart.html>. Accessed 1 April 2026.
- [3] Wazuh Documentation. *Installing the Wazuh server step by step*. Current documentation. Available at: <https://documentation.wazuh.com/current/installation-guide/wazuh-server/step-by-step.html>. Accessed 1 April 2026.
- [4] Wazuh Documentation. *Wazuh server cluster*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/wazuh-server-cluster/index.html>. Accessed 1 April 2026.
- [5] Wazuh Documentation. *Wazuh agent*. Current documentation. Available at: <https://documentation.wazuh.com/current/getting-started/components/wazuh-agent.html>. Accessed 1 April 2026.
- [6] Wazuh Documentation. *Log data analysis*. Current documentation. Available at: <https://documentation.wazuh.com/current/getting-started/use-cases/log-analysis.html>. Accessed 1 April 2026.
- [7] Wazuh Documentation. *Log data collection*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/index.html>. Accessed 1 April 2026.
- [8] Wazuh Documentation. *Configuring log collection for different operating systems*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/configuration.html>. Accessed 1 April 2026.
- [9] Wazuh Documentation. *localfile - Local configuration (ossec.conf)*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/localfile.html>. Accessed 1 April 2026.
- [10] Wazuh Documentation. *How it works - Log data collection*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/how-it-works.html>. Accessed 1 April 2026.
- [11] Wazuh Documentation. *Configuring syslog on the Wazuh server*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/syslog.html>. Accessed 1 April 2026.
- [12] Wazuh Documentation. *Wazuh manager*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/manager/wazuh-manager.html>. Accessed 1 April 2026.
- [13] Wazuh Documentation. *How it works - Monitoring system calls*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/capabilities/system-calls-monitoring/how-it-works.html>. Accessed 1 April 2026.

- [14] Wazuh Documentation. *MITRE ATT&CK framework*. Current documentation, Sysmon integration subsection. Available at: <https://documentation.wazuh.com/current/user-manual/ruleset/mitre.html>. Accessed 1 April 2026.
- [15] Wazuh Documentation. *Windows Defender logs collection*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/capabilities/malware-detection/win-defender-logs-collection.html>. Accessed 1 April 2026.
- [16] Wazuh Documentation. *JSON decoder*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/ruleset/decoders/json-decoder.html>. Accessed 1 April 2026.
- [17] Wazuh Documentation. *Cloud workload protection*. Current documentation. Available at: <https://documentation.wazuh.com/current/getting-started/use-cases/cloud-workload-protection.html>. Accessed 1 April 2026.
- [18] Wazuh Documentation. *Monitoring Amazon Web Services (AWS)*. Current documentation. Available at: <https://documentation.wazuh.com/current/cloud-security/amazon/index.html>. Accessed 1 April 2026.
- [19] Wazuh Documentation. *Monitoring Microsoft Azure with Wazuh*. Current documentation. Available at: <https://documentation.wazuh.com/current/cloud-security/azure/index.html>. Accessed 1 April 2026.
- [20] Wazuh Documentation. *Monitoring Google Cloud*. Current documentation. Available at: <https://documentation.wazuh.com/current/cloud-security/gcp/index.html>. Accessed 1 April 2026.
- [21] Wazuh Documentation. *Monitoring Office 365 audit logs*. Current documentation. Available at: <https://documentation.wazuh.com/current/cloud-security/office365/monitoring-office365-activity.html>. Accessed 1 April 2026.
- [22] Wazuh Documentation. *Monitoring GitHub*. Current documentation. Available at: <https://documentation.wazuh.com/current/cloud-security/github/index.html>. Accessed 1 April 2026.
- [23] Wazuh Documentation. *Decoders Syntax*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/decoders.html>. Accessed 1 April 2026.
- [24] Wazuh Documentation. *Rules Syntax*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html>. Accessed 1 April 2026.
- [25] Wazuh Documentation. *Custom rules*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html>. Accessed 1 April 2026.
- [26] Wazuh Documentation. *Alert management*. Current documentation. Available at: <https://documentation.wazuh.com/current/user-manual/manager/alert-management.html>. Accessed 1 April 2026.

- [27] Wazuh Documentation. *Navigating the Wazuh dashboard*. Current documentation, index-management section. Available at: <https://documentation.wazuh.com/current/user-manual/wazuh-dashboard/navigating-the-wazuh-dashboard.html>. Accessed 1 April 2026.
- [28] inLab FIB. *Wazuh – An Open Source platform unifying SIEM and XDR*. Published 27 May 2024. Available at: <https://inlab.fib.upc.edu/en/blog/wazuh-an-open-source-platform-unifying-siem-and-xdr>. Accessed 1 April 2026.